

---

**Name of Instruction: Add**

**Format: Add dst, src, trg**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:9] Cond; [8:6] Trg; [5:3] Src ; [2:0] Dst**

|      |        |        |       |       |       |
|------|--------|--------|-------|-------|-------|
| 00   | 000    | 2 bits | 3bits | 3bits | 3bits |
| TYPE | OPCODE | COND   | TRG   | SRC   | DST   |

**Purpose:**

Add 2 8-bit two's complement numbers and store the result.

**Description:**

The 8-bit register src is added to the 8-bit register trg and the result is stored inside of the destination register dst.

There is a possibility for the addition operation to overflow and if that occurs an overflow flag is set.

If the condition bits are set to 01, then an unsigned operation is performed and no overflow will be set.

**Operation:**

```
dst ← [src] + [trg]
If (src bit 8) == (trg bit 8) == sign and (dst ) == sign then
    dst ← result
Else If (src bit 8) != (trg bit 8) then
    dst ← result
Else
    dst ← result
    Overflow = 1
```

**Flags Set:**

FL - Overflow

---

**Name of Instruction: Sub**

**Format: Sub dst, src, trg**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:9] Cond; [8:6] Trg; [5:3] Src ; [2:0] Dst**

|      |        |        |       |       |       |
|------|--------|--------|-------|-------|-------|
| 00   | 000    | 2 bits | 3bits | 3bits | 3bits |
| TYPE | OPCODE | COND   | TRG   | SRC   | DST   |

**Purpose:**

Subtract 1 8-bit two's complement numbers from another and store the result.

**Description:**

The value in the 8 bit register trg is subtracted from the value in the 8 bit register src. The 8 bit two's complement result is stored in the register dst

If the operation result triggers an overflow, the overflow flag is set.

**Operation:**

```
dst ← [src] - [trg]
If (src bit 8) == (-) And (trg bit 8) == (+) and (dst) == (+) then
    dst ← result
    Overflow = 1
Else if (src bit 8) == (+) And (trg bit 8) == (-) and (dst) == (-) then
    dst ← result
    Overflow = 1
Else
    dst ← result
```

**Flags Set:**

FL - Overflow

---

**Name of Instruction: Negate**

**Format: Neg dst, src**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:9] Cond; [8:6] unused; [5:3] Src ; [2:0] Dst**

|      |        |        |         |       |       |
|------|--------|--------|---------|-------|-------|
| 01   | 001    | 2 bits | 3bits   | 3bits | 3bits |
| TYPE | OPCODE | COND   | NOTHING | SRC   | DST   |

**Purpose:**

Negate the value stored in a register and store the result in another register.

**Description:**

The 8-bit value stored in the src register is negated by taking the two's complement of the number. The result is then stored in the dst register.

**Operation:**

$\text{dst} \leftarrow (\text{Invert bits 1-8})[\text{src}] + 1$

**Flags Set:**

None

---

**Name of Instruction: Compare**

**Format: Cmp dst, src**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:9] Cond; [8:6] unused; [5:3] Src ; [2:0] Dst**

|      |        |        |         |       |       |
|------|--------|--------|---------|-------|-------|
| 01   | 100    | 2 bits | 3bits   | 3bits | 3bits |
| TYPE | OPCODE | COND   | NOTHING | SRC   | DST   |

**Purpose:**

A comparison function that takes two registers and compares their value to set flags registers for future operations. Or the result can be used to simply skip the next instruction.

**Description:**

The comparison function takes the 8-bit two's complement src register and makes a comparison with the 8-bit two's complement dst register.

If the value of src is greater than dst then the negative register is set to a negative value. This negative value can be examined for future operations. If not, the negative register is set to zero.

When the cond bits are greater than 00 the same operations occur but a greater src value will be used to skip the next instruction.

**Operation:**

If [src] > [dst] then  
    Negative ← (negative number)  
Else If [src] <= [dst] then  
    Negative ← 0

**Flags Set:**

FL - Negative register set regardless of outcome

---

**Name of Instruction: Branch (with conditions)**

**Format: Br Imm\_val**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:9] Cond; [8:0] Immediate Val**

|      |        |        |                 |
|------|--------|--------|-----------------|
| 11   | 100    | 2 bits | 9 bits          |
| TYPE | OPCODE | COND   | IMMEDIATE VALUE |

**Purpose:**

This instruction is used to either branch(jump) to a chosen instruction or to simply skip the next instruction.

**Description:**

This instruction's operation depends on the setting of the cond bits. If the cond bits are greater than 00 then the immediate value is ignored and the instruction flow will simply skip the next upcoming instruction.

When the cond bits aren't the operation may act as a jump to future instructions, with the immediate value to be used as an offset of the Instruction Pointer.

**Operation:**

```
If cond > 0
    Skip the next instruction
Else If cond == 0
    Skip the amount of instructions determined by the immediate
    value
```

**Flags Set:**

None

---

**Name of Instruction: Branch (Based on negative)**

**Format: Br zero, Imm\_val (Neg is implied)**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:8] Neg; [7:5] Zero; [4:0] Immediate Value**

|      |        |         |          |                 |
|------|--------|---------|----------|-----------------|
| 11   | 010    | 111     | 000      | 5 bits          |
| TYPE | OPCODE | NEG REG | ZERO REG | IMMEDIATE VALUE |

**Purpose:**

This instruction is used to branch based on the result of a previous Compare operation. Used to jump to different points in the program.

**Description:**

This instruction's operation depends on the results of a previous Compare statement. When the zero register is greater than the neg register, the condition is activated and control of the program jumps to that of the immediate value.

If the neg register is greater than or equal to the zero register, instruction flow continues as normal. The neg register is set to zero upon completion of instruction.

**Operation:**

```
If [neg] >= 0
    [Neg] ← 0
    Instruction flow continues as normal
Else If neg < 0
    [Neg] ← 0
    Instruction flow is determined by the immediate value
```

**Flags Set:**

FL - Negative register set to zero

---

**Name of Instruction: Console**

**Format: Con dst**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:8] Signal [7:0] Dst**

|      |        |       |                  |
|------|--------|-------|------------------|
| 11   | 111    | 3bits | 8 Bits - 1111111 |
| TYPE | OPCODE | DST   | SIGNAL VALUE     |

**Purpose:**

Outputs the data from the chosen register to the console.

**Description:**

The bit pattern signifies an output to the console. The value that is output is chosen from the 8-register specified in the dst field

**Operation:**

Console output ← [dst]

**Flags Set:**

None

---

**Name of Instruction: Load Immediate**

**Format: LI dst Imm\_val**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:8] Signal [7:0] Dst**

|      |        |       |                   |
|------|--------|-------|-------------------|
| 11   | 010    | 3bits | 8 Bits - 11111111 |
| TYPE | OPCODE | DST   | IMMEDIATE VALUE   |

**Purpose:**

Loads an 8-bit immediate value into a chosen register.

**Description:**

Operation takes the 8-bit immediate value specified in the instruction and stores it inside of the register determined by the destination field.

**Restrictions:**

Cannot set any of the special registers directly: Zero; Negative; Overflow

**Operation:**

$[dst] \leftarrow (8 \text{ bit Immediate Value})$

**Flags Set:**

None

---

**Name of Instruction: Load Immediate (half byte)**

**Format: LHI dst Imm\_val**

**Bit Representation:**

**[15:14] Type; [13:11] Opcode ; [10:8] Signal [7:0] Dst**

|      |        |       |                  |
|------|--------|-------|------------------|
| 11   | 011    | 3bits | 8 Bits - 1111111 |
| TYPE | OPCODE | DST   | IMMEDIATE VALUE  |

**Purpose:**

Loads an 4-bit immediate value into a chosen register.

**Description:**

Similar to the normal load immediate operation, this instruction takes a 4-bit immediate value specified in the instruction and stores it inside of the register determined by the destination field.

The 4-bits come from the rightmost 4 bits of the immediate value. The upper 4 bits are simply ignored

To maintain alignment, the 4th most bit is used to determine the sign extension of the value.

**Operation:**

$[dst] \leftarrow \text{Sign Extended (4 bit Immediate Value)}$

**Flags Set:**

None

---

**Benchmark Explanation:**

The benchmark programs used to test the design of the ISA are appropriate because they are able to showcase each instruction that would have to be implemented in the design as well as showcase the different variations some instructions could have.

Multiple instructions such as Compare or Branch (with condition) are able to have different operations depending on their bit pattern and the programs showcase these features and show how these variations can produce different outcomes, even with similar instructions, which was a part of the design choices that were made.

Both programs seemed to produce results inline with their initial design and intention.